

A Fast Algorithm for Constructing Trees from Distance Matrices

Joseph C. Culberson ^{*} Piotr Rudnicki [†]

November 15, 2001

Abstract

We present an algorithm which given a tree-realizable distance matrix constructs the tree in optimal $O(n^2)$ time. For trees of bounded degree k , the algorithm runs in $O(kn \log_k n)$ time, and for random trees apparently runs in $O(n)$ average time. We show how the algorithm can be used to test tree-realizability of a distance matrix.

Key words: Analysis of algorithms, trees, distance matrices.

1 Introduction

A distance matrix of order n is an $n \times n$ symmetric matrix D with non-negative entries $d_{ij} \in \mathcal{R}^+$ such that $d_{ii} = 0$ and $d_{ij} > 0$, for $1 \leq i \neq j \leq n$. When distance matrices are used to define finite metric spaces, then they must satisfy the triangle property, i.e. $d_{ij} \leq d_{ik} + d_{kj}$, for every $1 \leq i, j, k \leq n$.

Let $\mathcal{T} = \langle V, E, f \rangle$ be a weighted tree, where V is the set of vertices, E the set of edges, and $f: E \mapsto \mathcal{R}^+$ is the function assigning the weight (length) to each edge. The distance between $v_i, v_j \in V$ is the sum of weights of the edges on the unique path from v_i to v_j and is denoted by $d_{ij}^{\mathcal{T}}$. We say (after [6]) that $\mathcal{T}$ realizes D of order n if and only if $\{1, \dots, n\} \subseteq V$

^{*}Supported by Natural Sciences and Engineering Research Council Grant No. A-8053. Department of Computing Science, University of Alberta, Edmonton, Alberta, Canada, T6G 2H1. email: joe@alberta.uucp

[†]Supported in part by NSERC Grant No. A-9207. Department of Computing Science, The University of Alberta, Edmonton, Alberta, Canada T6J 2H1. email: piotr@alberta.UUCP.

and $d_{ij}^T = d_{ij}$, for $1 \leq i, j \leq n$. The vertices in $V_e = \{1, \dots, n\}$ are called *external*, and those in $V_i = V \setminus V_e$ are called *internal*, or *Steiner*, vertices. We say that D is tree-realizable if there exists a tree $\mathcal{T}$ that realizes D . If D is tree-realizable then by $\mathcal{T}(D)$ we mean the tree $\langle V, E, f \rangle$ that realizes D with the smallest $|V|$. Trivially, if D is tree-realizable then all Steiner vertices in $\mathcal{T}(D)$ have degree at least 3. It was proven by Hakimi and You [3] that if D is tree-realizable then $\mathcal{T}(D)$ is unique.

Several papers have been published on the characterization of distance matrices that are tree-realizable. Authors of these characterizations usually developed an algorithm that would construct the tree. Relatively little attention has been paid to the complexity of these algorithms.

Zaretskiĭ [7] gave necessary and sufficient conditions for the existence of $\mathcal{T}(D)$, when D is an integer matrix, and gave an algorithm for constructing the tree. His algorithm consists in finding the leaves of $\mathcal{T}(D)$, pruning them from D , and then recursing. The complexity of this algorithm is $O(n^4)$. Hakimi and Yau's [3] algorithm consists of adding one vertex at a time to the tree. Initially, cycles are created, and then reduced, if D is tree-realizable. The complexity of this algorithm is $O(n^3)$. Boesch [1] starts by finding the vertices that are at maximum distance in $\mathcal{T}(D)$ (we can find the maximum in $O(n)$ time). They must be leaves. Then he prunes the leaves from D in $O(n)$ time and recurses. The complexity of Boesch's algorithm is $O(n^2)$ independent of the topology of $\mathcal{T}(D)$. Patrinos and Hakimi [5] give another algorithm for constructing $\mathcal{T}(D)$ with complexity at least $O(n^2)$, but they do not give the details of the algorithm. Finally, Simões-Pereira and Zamfirescu [6] characterize tree-realizability in terms of submatrices of non-tree-realizable distance matrices. Their algorithm requires testing $\binom{n-1}{3}$ principal submatrices of order 4 of D , and thus is $O(n^3)$.

In this paper we present an $O(n^2)$ running time algorithm for constructing the tree from a tree-realizable distance matrix. We demonstrate that $\Omega(n^2)$ is the lower bound for any algorithm that constructs $\mathcal{T}(D)$, even if we do not count the time needed to input the n^2 entries of D . In this sense the algorithm of Boesch [1] is optimal. However, our algorithm outperforms Boesch's for many restricted topologies and in the average case.

The proposed algorithm can be adapted to test whether a given distance matrix D is tree-realizable. First, we attempt to construct the tree from D . The construction can fail and then D is not tree-realizable. If the construction succeeds, then we check whether the distances in the constructed tree are the same as in D . The complexity of the optimal algorithm for the latter is trivially $O(n^2)$.

2 The Algorithm

The algorithm for constructing a tree from a tree-realizable distance matrix is based on the following simple observations. For any three vertices in a tree, there is a unique vertex which lies on all three simple paths between pairs of vertices. Now, consider a path in the tree. The tree can be partitioned into subtrees that are ‘rooted’ at the vertices on the path. The distances between the end-points of the path and a given vertex uniquely determine the subtree into which the vertex belongs.

Construct a tree $\mathcal{T}$.

- INPUT. $n \geq 1$ — the number of external vertices in $\mathcal{T}$,
 $D[1..n, 1..n]$ — a tree-realizable distance matrix, $D[i, j] \in \mathcal{R}^+$,
 $D[i, j] > 0$ and $D[i, i] = 0$, for $1 \leq i \neq j \leq n$.
 OUTPUT. a weighted tree $\mathcal{T}$ with n vertices labeled $1, \dots, n$, and up to $n - 2$ vertices that are labeled with larger numbers (Steiner points), isomorphic with $\mathcal{T}(D)$.

We let $V_e = \{1, \dots, n\}$ be the set of external points. We let $S_p = \{n + 1, \dots, 2n - 2\}$, to give consistent identification to the created Steiner points (if any). By simple induction we have that the number of Steiner points in $\mathcal{T}(D)$ is at most $n - 2$ if D is a tree-realizable distance matrix of order n .

The computation is performed by calling the recursive procedure *Build_Tree* with the following three parameters:

1. $r \in V_e \cup S_p$. r is a vertex that has already been inserted into the constructed tree.
2. $u \subset V_e$. The vertices in u will be connected to the rest of $\mathcal{T}$ through r .
3. $d_r: u \mapsto \mathcal{R}^+$. d_r gives the distance from r to each element $i \in u$. Since r can be a Steiner point, the distance $\text{dist}(r, i)$ is not directly given by D . For uniformity, d_r is used even if $r \in V_e$.

The initial call to *Build_Tree* is set up as follows:

- 11 For some arbitrary $i \in V_e$, find $a \in V_e$ such that $D[i, a]$ is maximum. (Then there exists $b \in V_e$ such that $D[a, b]$ is maximum in D .) Set $u = V_e \setminus \{a\}$. Take the mapping $d_a: u \mapsto \mathcal{R}^+$ such that $d_a(i) = D[a, i]$ for each $i \in u$.

I2 $\mathcal{T} = \langle \{a\}, \emptyset, \emptyset \rangle$.

I3 *Build_Tree*(a, u, d_a)

Given any three external vertices i, j, k of a labeled tree, there is a single vertex which lies on all three paths (i, j) , (i, k) , and (j, k) . This vertex is called the *hub* of i, j, k (cf. [2]). By $hub(i, j, k)$ we will mean the distance from i to the *hub* of i, j, k ; $hub(i, j, k) = \frac{1}{2}(dist(i, j) + dist(i, k) - dist(j, k))$.

Procedure *Build_Tree* ($r \in V_e \cup S_p$; $u \subset V_e$; $d_r: u \mapsto \mathcal{R}^+$)

A1 If $u = \emptyset$ then return.

A2 Find $b \in u$ such that $d_r(b)$ is maximum.

A3 Partition $u \cup \{r\}$ into subsets u_k , $0 \leq k \leq m > 0$ such that $i, j \in u_k \iff hub(r, b, i) = hub(r, b, j)$. These constitute the vertices of subtrees on the path (r, b) .

A4 Call $i \in u_k$ *distinguished* if $hub(i, r, b) = 0$. In the following we compute the hub v_k , for each u_k .

For each $0 \leq k \leq m$ do

- (a) If $i \in u_k$ is distinguished, set $v_k = i$,
- (b) Otherwise the hub represents a Steiner point; remove the smallest s from S_p and set $v_k = s$.
- (c) Let $\bar{u}_k = u_k \setminus \{v_k\}$.

A5 For each $0 \leq k \leq m$ compute the function $d_{v_k}: \bar{u}_k \mapsto \mathcal{R}^+$ as follows:
 $d_{v_k}(j) = hub(j, r, b)$, for every $j \in \bar{u}_k$.

A6 Let $h_k = hub(r, b, v_k)$, for $0 \leq k \leq m$. Let π be the permutation such that $h_{\pi(k)} > h_{\pi(k-1)}$. For each $1 \leq k \leq m$, insert vertex $v_{\pi(k)}$ and edge $(v_{\pi(k-1)}, v_{\pi(k)})$ with weight $h_{\pi(k)} - h_{\pi(k-1)}$ into $\mathcal{T}$.

A7 Recursively call *Build_Tree*($v_k, \bar{u}_k, d_{v_k}$) for $0 \leq k \leq m$.

Note that the permutation mentioned in step A6 can be computed in step A3 by keeping the elements u_k of the partition sorted by distance of the hub from r , i.e. if $i \in u_k$ and $j \in u_{k+1}$ then $hub(r, b, i) < hub(r, b, j)$. This may improve the efficiency of step A3, depending on the chosen sorting technique.

Lemma 2.1 *If D of order $n \geq 1$ is tree-realizable then the tree construction algorithm terminates and outputs a weighted tree $\mathcal{T}$ isomorphic to $\mathcal{T}(D)$.*

Proof: The proof is by strong induction over the order of D .

Consider any call to $Build_Tree(r, u, d_r)$ in I3 or A7. Note that $r \notin u$ in virtue of I1 or A4c. The mapping d_r together with the entries $D[i, j]$ for $i, j \in u$ is another representation of a distance matrix for the vertices in $u \cup \{r\}$. Let $p = |u| + 1$. Assume that $Build_Tree$ satisfies the lemma for all $p < n$. We will show that the lemma then holds for $p = n$.

Case $n = 1$. The vertex labeled r has already been inserted into $\mathcal{T}$ (in I2 or A6) and it has to be present in $\mathcal{T}(D)$. Thus, if $n = 1$ the procedure returns immediately (A1).

Case $n > 1$. An external vertex $b \in u$ is chosen in A2. We will show that the sequence of vertices and edges inserted into $\mathcal{T}$ in step A6 is isomorphic to the path (r, b) in $\mathcal{T}(D)$. Since $\mathcal{T}(D)$ is a tree, the path (r, b) is unique. Let $(r, b) = (r = w_0, w_1, w_2, \dots, w_m = b)$ in $\mathcal{T}(D)$. Consider a vertex w_l , $1 \leq l \leq m$. The distance from r to w_l equals $hub(r, b, i)$ for some vertex $i \in \{1, \dots, n\}$. Thus we have that $d_{rw_l}^{T(D)} = h_{\pi(l)}$, where $h_{\pi(l)}$ is as in step A6, and $d_{w_l w_{l-1}}^{T(D)} = h_{\pi(l)} - h_{\pi(l-1)}$. If for some $j \in \{1, \dots, n\}$, $d_{rw_l}^{T(D)} = hub(r, b, j)$ then w_l is the external point j . If no such j exists then w_l is a Steiner point. The above is taken care of in steps A4a and A4b. Thus the sequence of weighted edges inserted by A6 to $\mathcal{T}$ is identical with the path (r, b) in $\mathcal{T}(D)$, up to the numbering of Steiner points.

$\mathcal{T}(D)$ can be partitioned into $m + 1$ subtrees s_k , $0 \leq k \leq m$ that are ‘rooted’ at $w_0, \dots, w_k, \dots, w_m$, respectively. The subtree s_k is connected through w_k to the path (r, b) . Steps A3 and A4c partition u such that $\bar{u}_k$ is the set of external points in s_k . The distance matrix D_k for s_k is partially stored in D , and partially in the corresponding mapping computed by A5 which gives the distances of the external points in s_k to w_k . If D is tree-realizable then D_k is tree-realizable.

For each $0 \leq k \leq m$ we have $|\bar{u}_k| < |u|$ because the vertex b is not in any of $\bar{u}_k$. $Build_Tree$ is recursively called in A7 for each s_k . By the inductive hypothesis the call $Build_Tree(v_k, \bar{u}_k, d_{v_k})$ constructs a tree isomorphic to s_k and connects it through v_k to the rest of $\mathcal{T}$. Note that v_k has already been inserted into $\mathcal{T}$. ■

Remark on testing tree-realizability of a distance matrix. Suppose D may not be tree-realizable. We modify the algorithm as follows. If any of the hub values in steps A3 or A5 turns out to be negative, then the algorithm

stops and outputs ‘no’. Indeed, if a *hub* value is negative then there exist $1 \leq a, b, c \leq n$ such that $D[a, c] > D[a, b] + D[b, c]$. This contradicts the triangle property required for a distance matrix to define a metric space. The algorithm also outputs ‘no’ when there is more than one distinguished vertex in step A4. We do not allow the external points to overlap.

If the modified algorithm outputs ‘no’, then D is not tree-realizable. Otherwise, we have to test whether distances among the external vertices in $\mathcal{T}$ are identical with entries in D . We compute an $n \times n$ matrix $D^{\mathcal{T}}$ and compare it with D . $d_{ij}^{\mathcal{T}}$ gives the length of the path (i, j) in $\mathcal{T}$, for $1 \leq i, j \leq n$. (The easy way of computing $D^{\mathcal{T}}$ in $O(n^2)$ time is not discussed here.) If D is tree-realizable then $D^{\mathcal{T}} = D$, because $\mathcal{T}$ is isomorphic to $\mathcal{T}(D)$. Therefore, if $D^{\mathcal{T}} \neq D$ then D is not tree-realizable.

3 Run Time Analysis

Throughout this section, we assume n is the number of external vertices. We refer to a tree as *topological* if all edges have weight one.

The problem has a trivial lower bound of $\Omega(n^2)$, if we consider input of the distance matrix. We assume throughout this section that the distance matrix is already in memory, thereby disallowing I/O costs.

3.1 Lower bounds

Consider the case of a tree with topology analogous to Figure 1a, for which all the non-zero entries in the distance matrix are identical except for d_{AB} and d_{BA} , which are smaller. In order to find the Steiner point C any algorithm has to access the entry d_{AB} (or d_{BA}). Thus any such algorithm must make $\Omega(n^2)$ accesses to D .

For the case of topological distances we may hope for an $O(n)$ complexity algorithm for topologies analogous to Figure 1a, since we can find a maximum entry in D in linear time. However, the case of topology as in Figure 1b requires us to find the minimum in each row of D in order to find the non-central Steiner points. Thus, to construct $\mathcal{T}(D)$ one has to examine $\Omega(n^2)$ entries in D . We summarize the above in

Lemma 3.1 *Construction of a tree from a distance matrix requires $\Omega(n^2)$ time in the worst case.*

It is worthwhile considering a lower bound for the case of a tree with exactly 2 leaves, which we call linear. Any construction algorithm takes

$\Omega(n \log n)$ time for this tree, assuming the comparison based model of sorting, since constructing it is equivalent to sorting the vertices by distance from the terminal vertices. However, if the linear tree is topological, then this lower bound need not apply as we can use a bucket sort to order the vertices. Variations on this observation will be used below to improve performance bounds when we have topological trees.

3.2 Optimality

We first observe that on the first call to *Build_Tree*, when b is found in step A2 the path (r, b) is a maximal path in the tree.

Lemma 3.2 *The construction algorithm requires at most $O(n^2)$ time to construct a tree on n vertices.*

Proof: The set up for the first call to *Build_Tree* trivially requires $O(n)$ time, so we need only prove our claim for *Build_Tree*. Although the algorithm is recursive, it is easier to analyze it using non-recursive techniques.

We note that on every call to *Build_Tree* in which $u \neq \emptyset$, there is at least one edge output in step A6. Since there are at most $2n - 2$ vertices including Steiner points in the tree, this implies that there are at most $2n - 3$ such calls (and at most $O(n)$ empty calls).

We compute an upper bound on the amount of work per edge output, keeping in mind that at least one edge is output per call. In one call to *Build_Tree* we can compute the cost as follows. Step A2 takes at most $O(n)$ time to find the maximum $d_r(b)$ over the set. Step A4 requires $O(n)$ since we consider each element in u once. Step A5 requires $O(n)$ time to compute the distance from each hub representative to each vertex in the corresponding partition. If, as suggested at the end of the algorithm, we maintain the partitions in order by increasing distance from r using any balanced tree implementation, then steps A3 and A6 can be done in $O(n \log k)$ time, where $k \geq 2$ is the number of classes in the partition. We output $k - 1$ edges in the call and so the cost per edge is

$$\frac{O(n) + O(n \log k)}{k - 1} = O(n)$$

Given the bound on the number of edges output, the lemma follows immediately. ■

3.3 Trees with bounded vertex degree

Note that for linear trees, on the first call to *Build_Tree*, r and b are the (only) leaves. Thus

Lemma 3.3 *Build_Tree constructs a linear tree in $O(n \log n)$ time.*

When we are dealing with a topological tree, the partition in step A3 can be maintained using bucket techniques, since given the distance to b , we know exactly how many and where the hubs of the elements of the partition are located. In this case it costs only constant time per vertex for step A3, and thus all steps require $O(n)$ time for the external call to *Build_Tree*.

Lemma 3.4 *Build_Tree constructs a topological linear tree in $O(n)$ time.*

The following lemma characterizes the worst case running time of the construction algorithm for the vast majority of topological trees.

Lemma 3.5 *For topological trees of bounded degree $k > 2$, the algorithm has a worst case time complexity of $O(kn \log_k n)$.*

Proof: The time required by the algorithm for topological trees is dominated by the number of *hub* computations in step A3 of *Build_Tree*. Since once a vertex, external or Steiner, is located on a path in the tree it no longer participates in such computations in other partitions, the number of computations is maximized when the longest path in every subtree is minimized. When the maximum degree is restricted, this leads to balanced trees where all internal vertices are of maximum degree. An example where the maximum degree is four, is shown in Figure 2.

To simplify the analysis, we will assume that the trees have $n = k(k - 1)^l$ external vertices, for integer l , with all external vertices being leaves. Thus, l is one less than the radius of a fully balanced tree. Throughout the following when we refer to the size of a subtree, we mean the number of external vertices in it. Other values of n will cause some longest paths to be (proportionally) longer than those in the following argument, and so the analysis will be an upper bound for all n .

The longest path in the tree must pass through the center of the tree, with $2l + 3$ vertices on it. The time required by *Build_Tree* on the center will dominate the running time. We analyze the running time of the first recursive call to *Build_Tree* on the subtree of size $(k - 2)(k - 1)^l$ on the

center vertex remaining after the initial path has been formed. (There are k branches each of equal size, and two are used for the initial path).

The longest path from the center divides the subtree into one subtree of size $(k-3)(k-1)^l$ remaining on the center vertex, l subtrees with sizes $(k-2)(k-1)^i$, $0 \leq i \leq l-1$, and one subtree with one external vertex. The one vertex subtrees induce no *hub* computations, and so can be ignored. The l subtrees other than the one remaining on the center can be treated recursively. The number of *hub* computations to compute this longest path will be $(k-2)(k-1)^l$. Note that we count the *hub* computation to locate b , that is, $hub(r, b, b)$.

Similarly, the next recursive call on the center vertex will require $(k-3)(k-1)^l$ computations, and produce one subtree of size $(k-4)(k-1)^l$, l subtrees of sizes $(k-2)(k-1)^i$, $0 \leq i \leq l-1$, and one subtree with one external vertex. Each recursive call on the center vertex in effect eliminates one more branch, and all the branches are of the same length. Thus, the time for the algorithm to complete the subtree on the center will be

$$T((k-2)(k-1)^l) = (k-1)^l \sum_{i=1}^{k-2} i + (k-2) \sum_{i=0}^{l-1} T((k-2)(k-1)^i), \quad l > 0$$

and $T(k-2) = \sum_{i=1}^{k-2} i$ (i.e. when $l = 0$). This recurrence is a simple full history recurrence in l , which can be reduced using standard techniques to

$$T((k-2)(k-1)^l) = l(k-1)^l \frac{(k-2)^2}{2} + (k-1)^l \frac{(k-1)(k-2)}{2}$$

Recalling that $n = k(k-1)^l$, we see that $T(n) = O(kn \log_k n)$. ■

Palmer [4], p. 118, cites results that more than 99.83% of the vertices have degree less than or equal 7 for large unlabeled random trees. Labeled random trees will have even higher percentages of low degree vertices. Simulations, in which Prüfer sequences [4] have been used to generate random labeled trees, suggest linear average time for building topological trees using our algorithm.

4 Conclusion

We have shown an optimal $O(n^2)$ algorithm to construct and test a tree given a tree realizable distance matrix. We have also shown that if a tree-realizable distance matrix is available, then constructing the tree for a large

variety of tree configurations can be done in subquadratic time in the worst case.

Note added in proof

When this paper was in review, it came to our attention that V. Bagatelj, T. Pisanski, and J. M. S. Simões-Pereira reported an $O(n^2)$ algorithm for “Tree-Realizability of Distance Matrices” (abstract) on the *SIAM Conference on Discrete Mathematics*, San Francisco, 13–16 June 1988. Their algorithm appears similar to ours; however, at the time when this paper was submitted (September 8, 1988) the full version of their paper was not available.

References

- [1] F. T. Boesch. Properties of the distance matrix of a tree. *Q. Appl. Math.*, 26:607–609, 1968.
- [2] S. Chaiken, A. K. Dewdney, and P. J. Slater. An optimal diagonal tree code. *SIAM J. Algebraic Discrete Methods*, 4(1):42–49, 1983.
- [3] S. L. Hakimi and S. S. Yau. Distance matrix of a graph and its realizability. *Q. Appl. Math.*, 22:305–317, 1964.
- [4] E. M. Palmer. *Graphical Evolution. Wiley-Interscience Series in Discrete Mathematics*, John Wiley & Sons, Inc., 1985.
- [5] A. N. Patrinos and S. L. Hakimi. The distance matrix of a graph and its tree realization. *Q. Appl. Math.*, 30:255–269, Oct. 1972.
- [6] J. M. S. Simões-Pereira and C. M. Zamfirescu. Submatrices of non-tree-realizable distance matrices. *Linear Algebra Appl.*, 44:1–17, 1982.
- [7] K. A. Zaretskiĭ. Postroenie dereva po naboru rasstoianĭĭ mezhdu visitchimi verшинami. *Uspekhi Matematicheskikh Nauk*, 20(6):90–92, 1965.

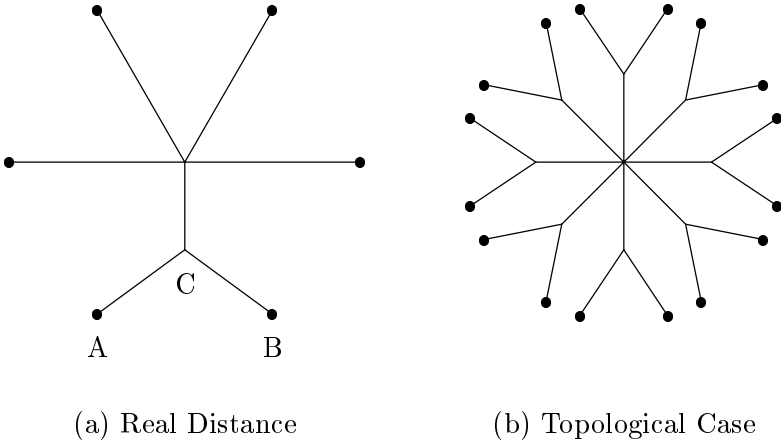


Figure 1: Trees Which Exhibit the $\Omega(n^2)$ Lower Bounds

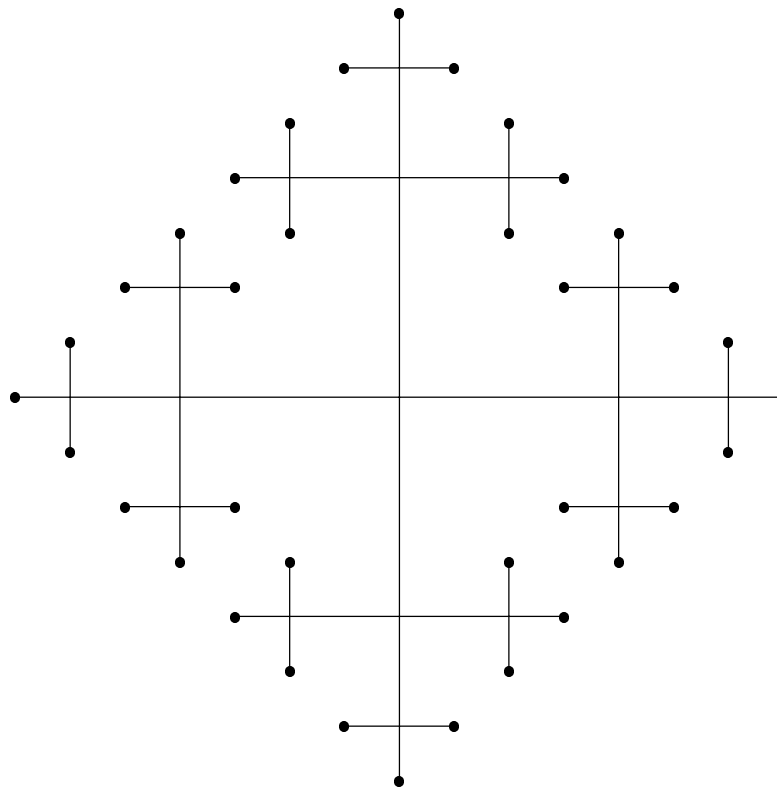


Figure 2: The Bushiest Graph of Degree Four and Radius 3